## Task 1.1: Trace Bubble Sort Logic

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Trace Bubble Sort manually on the array: `arr = [5, 2, 9, 1, 5, 6]`

```
start          [5, 2, 9, 1, 5, 6]

-- i=0  (j runs 0..4) --
i=0 j=0  5 > 2   SWAP   [2, 5, 9, 1, 5, 6]
i=0 j=1  5 <= 9  keep   [2, 5, 9, 1, 5, 6]
i=0 j=2  9 > 1   SWAP   [2, 5, 1, 9, 5, 6]
i=0 j=3  9 > 5   SWAP   [2, 5, 1, 5, 9, 6]
i=0 j=4  9 > 6   SWAP   [2, 5, 1, 5, 6, 9]     <- 9 locked at index 5

-- i=1  (j runs 0..3) --
i=1 j=0  2 <= 5  keep   [2, 5, 1, 5, 6, 9]
i=1 j=1  5 > 1   SWAP   [2, 1, 5, 5, 6, 9]
i=1 j=2  5 <= 5  keep   [2, 1, 5, 5, 6, 9]     <- equal, no swap (stable)
i=1 j=3  5 <= 6  keep   [2, 1, 5, 5, 6, 9]     <- 6 locked at index 4

-- i=2  (j runs 0..2) --
i=2 j=0  2 > 1   SWAP   [1, 2, 5, 5, 6, 9]
i=2 j=1  2 <= 5  keep   [1, 2, 5, 5, 6, 9]
i=2 j=2  5 <= 5  keep   [1, 2, 5, 5, 6, 9]     <- 5 locked at index 3

-- i=3  (j runs 0..1) --
i=3 j=0  1 <= 2  keep   [1, 2, 5, 5, 6, 9]
i=3 j=1  2 <= 5  keep   [1, 2, 5, 5, 6, 9]
no swaps in pass i=3 -> swapped stays False -> break

final          [1, 2, 5, 5, 6, 9]

Total comparisons: 14
Total swaps:        6
```

## 1.2 Insertion Sort Logic and Invariants

```python
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]  # Shift right
            j -= 1
        arr[j + 1] = key         # Place key
    return arr
```

Trace Insertion Sort manually on the array: `arr = [7, 3, 5, 8, 2]`

```
start          [7, 3, 5, 8, 2]        sorted prefix [7]

(key is pulled out into a variable, so the array shows a duplicate
 value while shifting until the key is placed back in)

-- i=1  key=3  (j starts at 0) --
i=1 j=0   7 > 3   shift 7 right   [7, 7, 5, 8, 2]
i=1 j=-1  j < 0, stop
          place key at j+1=0      [3, 7, 5, 8, 2]   <- prefix [3, 7]           (1 cmp,

-- i=2  key=5  (j starts at 1) --
i=2 j=1   7 > 5   shift 7 right   [3, 7, 7, 8, 2]
i=2 j=0   3 <= 5  stop
          place key at j+1=1      [3, 5, 7, 8, 2]   <- prefix [3, 5, 7]        (2 cmp,

-- i=3  key=8  (j starts at 2) --
i=3 j=2   7 <= 8  stop
          place key at j+1=3      [3, 5, 7, 8, 2]   <- prefix [3, 5, 7, 8]     (1 cmp,

-- i=4  key=2  (j starts at 3) --
i=4 j=3   8 > 2   shift 8 right   [3, 5, 7, 8, 8]
i=4 j=2   7 > 2   shift 7 right   [3, 5, 7, 7, 8]
i=4 j=1   5 > 2   shift 5 right   [3, 5, 5, 7, 8]
i=4 j=0   3 > 2   shift 3 right   [3, 3, 5, 7, 8]
i=4 j=-1  j < 0, stop
          place key at j+1=0      [2, 3, 5, 7, 8]   <- prefix [2, 3, 5, 7, 8]  (4 cmp,

final          [2, 3, 5, 7, 8]

Total comparisons: 8
Total shifts:      6
```

# Task 1.3: Sorting Analysis Questions

```
TODO 1.3A (Inversions & Shifts):
List all inversions (pairs of indices (i, j) where i < j and arr[i] > arr[j])
in the initial array [7, 3, 5, 8, 2]:
A:
- Inversions: checked all 10 pairs with i < j in [7, 3, 5, 8, 2] (indices 0..4)
    (0,1): 7 > 3
    (0,2): 7 > 5
    (0,4): 7 > 2
    (1,4): 3 > 2
    (2,4): 5 > 2
    (3,4): 8 > 2
- Total number of inversions: 6
- Does this total exactly equal the number of shifts you counted in Task 1.2? (Yes/No):

TODO 1.3B (Early Stopping Flag):
Why does Bubble Sort require an explicit boolean flag (`swapped`) to achieve
O(N) best-case time on sorted data, whereas Insertion Sort naturally achieves O(N) with
A: Insertion Sort doesn't need a flag because if its all in order, then arr[j]>key is a
so the while loop can finish immediately, and the for loop can therefore finish in O(n)
the same arr. Bubble sort will keep going without a flag even whens sorted, because the
and will continue all the way until the end, checking everything even if already sorted

TODO 1.3C (Stability):
If a programmer changes line 33 of Bubble Sort to `if arr[j] >= arr[j + 1]:`,
does the algorithm still produce a sorted array? Does it remain stable? Explain why or
A: Still sorted: Yes. Swapping two equal values does not change the order of the values
   and anything bigger than its neighbor still gets swapped right, so the largest eleme
   still bubbles to the end on every pass and the final array is in order.

   Still stable: No. With >= two equal neighbors now swap places. Example with tags:
   [5a, 5b] -> 5a >= 5b is True -> swap -> [5b, 5a]. The equal elements crossed each ot
   so their original left-to-right order is lost.
```

# Part 2: Lomuto Partition Scheme

## Task 2.1: Trace Lomuto Partition Scheme

Trace Lomuto partition on `arr = [2, 8, 7, 1, 3, 5, 6, 4]` with `low = 0, high = 7` (Pivot = $arr[7] = 4$).

```
start   [2, 8, 7, 1, 3, 5, 6, 4]   pivot = 4   i = -1

j=0  arr[0]=2  2 <= 4  yes  i -> 0, swap arr[0],arr[0]   [2, 8, 7, 1, 3, 5, 6, 4]
j=1  arr[1]=8  8 <= 4  no   i stays 0                    [2, 8, 7, 1, 3, 5, 6, 4]
j=2  arr[2]=7  7 <= 4  no   i stays 0                    [2, 8, 7, 1, 3, 5, 6, 4]
j=3  arr[3]=1  1 <= 4  yes  i -> 1, swap arr[1],arr[3]   [2, 1, 7, 8, 3, 5, 6, 4]
j=4  arr[4]=3  3 <= 4  yes  i -> 2, swap arr[2],arr[4]   [2, 1, 3, 8, 7, 5, 6, 4]
j=5  arr[5]=5  5 <= 4  no   i stays 2                    [2, 1, 3, 8, 7, 5, 6, 4]
j=6  arr[6]=6  6 <= 4  no   i stays 2                    [2, 1, 3, 8, 7, 5, 6, 4]
end  swap arr[i+1]=arr[3] with arr[high]=arr[7]          [2, 1, 3, 4, 7, 5, 6, 8]
return i + 1 = 3
```

```
Resulting Left Subarray (<= 4): [2, 1, 3]        (indices 0..2)
Resulting Pivot Index and Value: index 3, value 4
Resulting Right Subarray (> 4): [7, 5, 6, 8]     (indices 4..7)
```

# Task 2.2: Lomuto Duplicate Analysis

```
TODO 2.2A:
If arr = [5, 5, 5, 5, 5] is partitioned using Lomuto partition (pivot = 5):
- What will the final value of i be at the end of the loop?
- What index will the pivot end up at?
- What are the sizes of the two recursive subproblems passed to quicksort?
A: Every arr[j] is 5 and 5 <= 5 is True, so i goes up by one on every step of the loop
   (j = 0, 1, 2, 3). Each "swap" is arr[i] with itself, so the array never changes.
   - Final i = 3 (that is n - 2, one less than the last index).
   - Pivot ends up at i + 1 = 4, which is high, the very last index.
   - Left subproblem arr[0..3] has size 4 (n - 1). Right subproblem is empty, size 0.
   This is the worst possible split. Each level of recursion only removes one element,
   so quicksort takes N levels and O(n^2) total time on identical elements.

TODO 2.2B:
Run `python lomuto_partition.py` to view DEMO 3.
How does Hoare partition partition the array [5, 5, 5, 5, 5]?
What are the resulting subproblem sizes?
A: Hoare uses two pointers moving toward each other, and it stops each pointer on an el
   that is equal to the pivot instead of skipping it. So on [5, 5, 5, 5, 5] both pointe
   stop immediately, swap (5 with 5, no visible change), and move inward:
     swap #1: arr[0] and arr[4]
     swap #2: arr[1] and arr[3]
   The pointers cross at i = 2, j = 2, and the split index is 2.
   - Left subproblem arr[0..2] has size 3.
   - Right subproblem arr[3..4] has size 2.
```

# Part 3: Quicksort Benchmarks (1,000,000 Elements & Edge Cases)

## Task 3.1 & 3.2: Benchmark Observations & Analysis

```
TODO 3.1:
Record your execution times for n = 1,000,000 random integers:
- Python Timsort:         0.115 s   (1.0x)
- Lomuto (Random Pivot):  0.931 s   (8.1x)
- Lomuto (Median-of-3):   0.799 s   (6.9x)
- Lomuto (Last Element):  0.823 s   (7.1x)
- Hoare (Two-Pointer):    0.758 s   (6.6x)
- 3-Way Quicksort:        1.156 s  (10.0x)
```

On random data all the quicksorts are in the same ballpark, since a random array gives roughly balanced splits no matter how the pivot is picked. Timsort wins because it is written in C, not because of a better big-O.

Edge case times (n = 10,000 unless noted):

| Scenario | Lomuto (Last) | Lomuto (Random) | Lomuto (Med-of-3) | Hoare | 3-Way | Timsort |
|---|---|---|---|---|---|---|
| Already Sorted | 1.3326 s | 0.0051 s | 0.0036 s | 0.0030 s | 0.0063 s | 0.0000 s |
| Reverse-Sorted | 0.9268 s | 0.0053 s | 0.0059 s | 0.0030 s | 0.0064 s | 0.0000 s |
| All Identical | 1.1959 s | 1.2041 s | 1.1922 s | 0.0035 s | 0.0002 s | 0.0000 s |
| Few Unique (n=20,000) | 0.4847 s | 0.4895 s | 0.4984 s | 0.0079 s | 0.0019 s | 0.0008 s |
| Nearly Sorted (n=20,000) | 0.0554 s | 0.0112 s | 0.0158 s | 0.0063 s | 0.0131 s | 0.0002 s |

TODO 3.2A:
In the edge case benchmarks:
What happens to Lomuto with Last Pivot on an Already Sorted array? Why?
How does Random Pivot or Median-of-3 fix this?
A: It blows up. Lomuto with the last element as pivot took 1.33 s on a sorted array of
   only 10,000 elements, while random pivot took 0.005 s and median-of-3 took 0.004 s.
   That is about 300x slower for the same input.

   Why: on a sorted array the last element is the biggest one. Every other element is
   <= the pivot, so i moves up on every step and the pivot ends up staying at the end.
   of recursion doing about n work each, so O(n^2) instead of O(n log n).

   Fix: random pivot picks a random element and swaps it to the end before partitioning
   On a sorted array a random element is probably somewhere in the middle of the values
   so the split is roughly even. Median-3 is similar, on a sorted array it is the media

TODO 3.2B:
On the "All Identical Elements" scenario:
Compare the runtime of Lomuto Quicksort vs. Hoare Quicksort vs. 3-Way Quicksort.
Explain why Hoare and 3-Way Quicksort avoid Lomuto's quadratic explosion on identical d
A: Times on 10,000 identical elements:
   - Lomuto (last pivot):    1.196 s
   - Lomuto (random pivot):  1.204 s
   - Lomuto (median-of-3):   1.192 s
   - Hoare:                  0.0035 s
   - 3-Way:                  0.0002 s
   All three Lomuto versions are equally bad here. Changing the pivot does not help,
   because every element is still the pivot value. The test arr[j] <= pivot is always T

   Hoare avoids it because it stops both pointers on elements equal to the pivot and sw
   them. On identical data the pointers each move one step at a time from both ends and
   meet in the middle meaning O(n log n).

   3-Way avoids it even better. It splits the array into three parts: less than pivot,
   equal to pivot, greater than pivot. Only the less and greater parts get recursed on.
   On identical data everything lands in the equal part, both other parts are empty, an
   there is nothing left to recurse on. One pass and done, so O(n).

TODO 3.2C:
Why does standard recursive Quicksort risk crashing with `RecursionError` in Python
when sorting large unbalanced arrays, and how does tail-recursion elimination
or small-side recursion prevent this?
A: Python limits how deep function calls can nest.

```
Normal quicksort recurses on both halves. When the splits are unbalanced (n-1 and 0),
each recursive call only shrinks the problem by one,
   so the recursion depth is n. For n = 10,000 that is 10,000 nested calls, way past th
   limit, so Python raises RecursionError before it finishes.

   The fix is to only recurse on the smaller side, and handle the bigger side with a lo
   instead of a recursive call (the tail-recursion elimination).

   The smaller side is always at most half the array, so every recursive call at least
   halves the size. That caps the depth at about log2(n).
   The loop does not use any stack space, so the big side is free.
   This won't cause the recursion error to trigger.
```

---

# Part 4: Representing Binary Trees via Arrays

## Task 4.1: Manual Tree Traversal Trace

`arr = [50, 30, 20, 15, 10, 8, 16]` (0-indexed)

```
                50          index 0
              /    \
            30      20      index 1, 2
           /  \    /  \
         15   10  8   16    index 3, 4, 5, 6

left child of i  = 2i + 1
right child of i = 2i + 2
```

```
Pre-Order Traversal  (Root -> Left -> Right): [50, 30, 15, 10, 20, 8, 16]
In-Order Traversal   (Left -> Root -> Right): [15, 30, 10, 50, 8, 20, 16]
Post-Order Traversal (Left -> Right -> Root): [15, 10, 30, 8, 16, 20, 50]
Level-Order Traversal (Breadth-First):        [50, 30, 20, 15, 10, 8, 16]
Is this array a valid Max-Heap? (Yes/No):     [ Yes ]
```

## Task 4.2: Array Tree Practice Code

Done in `array_tree_practice.py`.

# Revised Lab 2

## Part 1: Sorting trace additions

### 1.1 Guaranteed sorted suffixes

The comparison-by-comparison Bubble Sort trace is in Task 1.1 above. The guaranteed sorted suffix after each pass is:

| Pass | Guaranteed sorted suffix |
|---|---|
| 1 | `[9]` |
| 2 | `[6, 9]` |
| 3 | `[5, 6, 9]` |
| 4 | `[5, 5, 6, 9]` |

There are 14 comparisons and 6 swaps. Pass 4 makes no swaps, so the entire array is sorted and the early-exit check stops the algorithm.

### 1.3A Sorted-input running time

On sorted input, Bubble Sort makes one pass with `n - 1` comparisons and no swaps, then exits. Insertion Sort makes one failed element comparison per iteration and no shifts. Both take O(n) time.

Without Bubble Sort's early-exit check, it performs every pass even on sorted input: `(n - 1) + (n - 2) + ... + 1 = n(n - 1)/2` comparisons. Its best-case time becomes $\Theta(n^2)$.

### 1.3B Stability

Strict > comparisons leave equal-key items in their original order. Bubble Sort does not swap equal neighbors. Insertion Sort stops shifting when it reaches an equal value and places the saved key after it.

Using >= in Bubble Sort still produces sorted numeric values, but it is not stable: `[5A, 5B]` becomes `[5B, 5A]` after the equal neighbors swap.

## Part 2: Lomuto additions

### 2.2 Implementation

Implemented `lomuto_partition` in `quicksort_practice.py`.

### 2.3A Pivot scanning and final swap

The pivot stays at `arr[high]` while the scanning loop partitions the other elements. After the loop, `low` through `i` contains values at most the pivot, and `i + 1` through `high - 1` contains values greater than it. The final swap puts the pivot at `i + 1`, between those regions, so that position can be excluded from later recursive calls.

### 2.3B Duplicate and ascending inputs

For `[5, 5, 5, 5]`, every scanned value is at most the pivot. The final `i` is `3`, the returned pivot index is `4`, and the left and right subproblem sizes are `4` and `0`.

Repeating this split removes only one element per level. The partition comparisons add up to `(n - 1) + (n - 2) + ... + 1`, giving $\Theta(n^2)$ time.

Ascending, distinct values produce the same subproblem sizes, `n - 1` and `0`, because the last-element pivot is always the largest remaining value.

# Part 3: Merge Sort

## 3.1A Splitting and merging

| Stage | Lists |
|---|---|
| Start | `[7, 2, 6, 3]` |
| Split into halves | `[7, 2]` and `[6, 3]` |
| Split into single elements | `[7]`, `[2]`, `[6]`, `[3]` |
| Merge each pair | `[2, 7]` and `[3, 6]` |
| Final merge | `[2, 3, 6, 7]` |

## 3.1B Merge trace

Merge `[2, 5, 8]` and `[1, 5, 9]`. The indices below are their values before each comparison.

| `i` | `j` | Values compared | Take from | Result so far |
|---|---|---|---|---|
| 0 | 0 | 2 and 1 | Right | `[1]` |
| 0 | 1 | 2 and 5 | Left | `[1, 2]` |
| 1 | 1 | 5 and 5 | Left | `[1, 2, 5]` |
| 2 | 1 | 8 and 5 | Right | `[1, 2, 5, 5]` |
| 2 | 2 | 8 and 9 | Left | `[1, 2, 5, 5, 8]` |

The left list is exhausted with `i = 3` and `j = 2`. Append the remaining 9 from the right list. The final result is `[1, 2, 5, 5, 8, 9]`.

## 3.2 Implementation

Implemented `merge` in `mergesort_practice.py`.

## 3.2A Remaining elements

The comparison loop stops as soon as either input is exhausted, even if the other input still has elements. Those remaining elements are already sorted and belong after the selected values, so they must be copied into the result. Returning immediately would lose them, such as the final 9 in the trace above.

# Part 4: Min-heaps and Heapsort

## 4.1 Array representation

For `[4, 10, 8, 30, 15, 20, 16]`:

| Requested relationship | Index calculation | Index | Value |
|---|---|---|---|
| Left child of index 1 | `2 * 1 + 1` | 3 | 30 |
| Right child of index 1 | `2 * 1 + 2` | 4 | 15 |
| Parent of index 6 | `(6 - 1) // 2` | 2 | 8 |

The leaf indices are 3, 4, 5, and 6, since none has a child index below 7.

This is a min-heap: `4 <= 10`, `4 <= 8`, `10 <= 30`, `10 <= 15`, `8 <= 20`, and `8 <= 16`. Every parent is at most as large as each of its children.

## 4.2A Sift-down trace

Start with `[25, 10, 8, 30, 15, 20, 16]`, `i = 0`, and `heap_size = 7`.

| Step | Current `i` | Smaller child | Action | Array afterward |
|---|---|---|---|---|
| 1 | 0 | Index 2, value 8 | Swap indices 0 and 2 | `[8, 10, 25, 30, 15, 20, 16]` |
| 2 | 2 | Index 6, value 16 | Swap indices 2 and 6 | `[8, 10, 16, 30, 15, 20, 25]` |
| 3 | 6 | None | Stop | `[8, 10, 16, 30, 15, 20, 25]` |

We choose the smaller child because moving the larger child up would leave it greater than its smaller sibling, violating the min-heap property. The process stops at index 6 because its child indices, 13 and 14, are outside the active heap.

## 4.2B and 4.3A Implementations

Implemented `min_heapify_down` and the extraction loop of `heap_sort` in `heapsort_practice.py`.

## 4.3B One Heapsort extraction

Start with `[4, 10, 8, 30, 15, 20, 16]`.

| Step | Full array | Active heap size | Sorted suffix |
|---|---|---|---|
| Swap root with index 6 | `[16, 10, 8, 30, 15, 20, 4]` | 6 | `[4]` |
| Sift down: swap indices 0 and 2 | `[8, 10, 16, 30, 15, 20, 4]` | 6 | `[4]` |

At index 2, the only active child is index 5, with value 20. Since `16 <= 20`, sift-down stops. The final active heap is `[8, 10, 16, 30, 15, 20]`, with sorted suffix `[4]`.

The suffix must be excluded because its values are already in their final positions. Including the 4 in sift-down could move that minimum back into the active heap and undo the extraction.